SchemeSathi: Scholarship Eligibility Navigator

Complete Technical Architecture, OOP Implementation Mapping & Defense Guide

Author / Lead Developer: Priyanshu Gupta	**Academic Domain:** Object-Oriented Software Design
Backend Architecture: Java 17 + Maven + Embedded HTTP Server	**Security & Cryptography:** SHA-256 + 16-Byte Secure Salt Hashing
Frontend Framework: React 19 + Vite (Modern Dark/Light System)	**Cloud Deployment:** Multi-Stage Docker on Render (HTTPS Enabled)

1. Project Overview & Problem Statement

SchemeSathi is an automated scholarship eligibility and navigation system engineered to solve the complex discovery problem faced by Indian students. Instead of manually inspecting numerous disparate portals and government gazettes, students provide their academic performance (marks %), annual family income, and demographic category. The Java OOP rule engine executes polymorphic rule verification to output instant eligibility outcomes, detailed disqualification reasons, mandatory document checklists, and downloadable plain-text audit reports.

2. Deep-Dive: The 4 Core OOP Principles in SchemeSathi

The project is structured strictly around the four fundamental pillars of Object-Oriented Programming:

OOP Pillar	Source Code Location	Implementation & Architectural Purpose
1. Abstraction	`com.schemesathi.model.Scheme` `com.schemesathi.report.ReportWriter`	• Scheme is an **abstract class** defining common attributes (id, name, required documents) and abstract methods isEligible(Student s) and getCriteriaDescription(). • ReportWriter is an **interface** hiding underlying File I/O behind an abstract contract.

2. Encapsulation	`com.schemesathi.model.Student`
	`com.schemesathi.auth.User`
	`com.schemesathi.auth.AuthService`
		• All class fields are declared private. Data mutation occurs only through getters/setters with strict defensive validation (e.g., marks between 0-100%, non-negative income).
		• Passwords are encrypted with SHA-256 and unique 16-byte random salts before storage.

3. Inheritance	`MeritScheme extends Scheme`
	`IncomeScheme extends Scheme`
	`CategoryScheme extends Scheme`
		• Concrete subclasses inherit shared fields and constructors via super(...) while introducing specialized criteria (e.g. minimumMarks in MeritScheme, maximumAnnualIncome in IncomeScheme).

4. Polymorphism	`com.schemesathi.service.EligibilityService`	• **Dynamic Method Dispatch**: The evaluation engine handles a generic collection List<Scheme> and calls scheme.isEligible(student). The JVM resolves the concrete implementation at runtime without instanceof checks, adhering to the **Open/Closed Principle**.

3. Runtime Polymorphism in Action (Code Proof)

4. Project Layered Structure & Clean Architecture

Layer / Package	Primary Responsibility & Classes
`com.schemesathi.model`	Domain entities: Student, abstract Scheme, MeritScheme, IncomeScheme, CategoryScheme.
`com.schemesathi.service`	Business rule evaluation: EligibilityService running polymorphic verification and breakdown analysis.
`com.schemesathi.loader`	JSON persistence ingestion: SchemeLoader reading scholarship rules from data/schemes.json.
`com.schemesathi.report`	Export subsystem: ReportWriter (interface) and EligibilityReportWriter generating formatted text dossiers.
`com.schemesathi.auth`	Security, hashing & auditing: AuthService, User, LoginRecord with SHA-256 + 16-byte random salt.
`com.schemesathi.server`	Microserver layer: ApiServer exposing REST endpoints (/api/evaluate, /api/auth, /api/admin).
`frontend/`	Client-side SPA: React 19 + Vite with Light/Dark CSS variables, WebGL2 animations, and isolated Admin Console.

5. Key Viva & Presentation Defense Questions

Q1: Why did you use an abstract class instead of an interface for Scheme?

Model Answer: An abstract class allows sharing state (fields like id, name, requiredDocuments) and shared methods (getters/setters) across all schemes via super(...). Interfaces cannot hold instance fields. Abstract methods (isEligible) ensure each subclass provides custom evaluation logic.

Q2: How does the system handle security and passwords?

Model Answer: Passwords are never stored in plaintext. In AuthService.java, every password is combined with a cryptographically secure 16-byte random salt generated via java.security.SecureRandom and hashed with SHA-256. This prevents rainbow table attacks and protects credentials.

Q3: How does the client communicate with the Java backend?

Model Answer: The Java backend runs an embedded HttpServer on port 8080 (or cloud $PORT) exposing REST endpoints. It implements CORS headers (Access-Control-Allow-Origin: *) allowing the React frontend to asynchronously POST student data and receive structured JSON responses.

Q4: How was this project tested?

Model Answer: We implemented 24 comprehensive automated unit tests using JUnit 5 in src/test/java. Every test verifies domain constraints, polymorphic evaluations, report generation, and credential hashing. All 24 tests pass cleanly during mvn test.

Q5: How is the application deployed to production?

Model Answer: We created a production-ready multi-stage Dockerfile compiling the Java Fat-JAR and serving it on Render Cloud with automatic HTTPS. The React frontend is deployed as a high-performance Static Site on Render connected to the cloud backend.